

OpenFLASH: An open-source flexible library for analytical and semi-analytical hydrodynamics calculations

Hope Best^{1,3}, Kapil Khanal^{1,3}, Rebecca McCabe^{1,3}, Ruiyang Jiang^{1,3}, Collin Treacy^{2,3}, and Maha Haji^{2,3}¶

¹ Cornell University, Ithaca, NY, United States^{ROR} ² Department of Mechanical Engineering, University of Michigan, United States^{ROR} ³ Symbiotic Engineering and Analysis (SEA) Lab ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- [Review](#) ↗
- [Repository](#) ↗
- [Archive](#) ↗

Editor: ↗

Submitted: 27 October 2025

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#))

Summary

OpenFLASH is a Python package for solving hydrodynamic boundary value problems using analytical and semi-analytical methods. It implements the Matched Eigenfunction Expansion Method (MEEM) for bodies of multiple concentric cylinders. This method, presented by McCabe et al. (2024) at the UMERC+METS 2024 Conference, can reduce the runtime by two orders of magnitude compared to traditional Boundary Element Method (BEM) solvers, making it more suitable for design optimization studies of floating structures such as wave energy converters (WECs).

Statement of Need

Wave energy converters (WECs) hold significant promise for transforming wave oscillation into energy, offering predictability and energy security that complement other renewable sources (Bhattacharya et al., 2021; Fusco et al., 2010). However, the optimization of WECs has been hindered by the computational costs of modeling their hydrodynamic interactions in waves. OpenFLASH addresses this challenge by providing an open-source, object-oriented Python implementation for modeling hydrodynamic forces using semi-analytical methods.

The package is designed to handle multi-domain problems, including exterior domains extending to infinity and interior domains with specific radial extents. The computational workflow begins by defining the geometry, assembling and solving the linear system, and calculating hydrodynamic coefficients. This specialization bridges the gap between purely analytical derivations and heavy numerical computation, offering a tool that is both modular and extensible for marine engineering research.

State of the Field

Hydrodynamic modeling typically relies on BEM solvers like WAMIT or Capytaine (Ancellin & Dias, 2019), which require surface meshing and become computationally expensive during wide-frequency sweeps. Alternative tools like MDOcean (McCabe et al., 2026) and SAHydro (Olaya et al., 2015) utilize MEEM but are largely restricted to specific 2- or 3-cylinder configurations within the MATLAB ecosystem. Other specialized tools like Hulme_Heaving_Sphere (Choi, 2019) are limited to non-cylindrical classes of devices.

OpenFLASH was built as a standalone library because existing BEM-based codes like Capytaine

use a fundamentally different computational architecture (surface integration vs. domain decomposition). While MATLAB-based scripts exist for MEEM, OpenFLASH abstracts this logic into a modular API supporting arbitrary stepped-cylindrical geometries. This fills a critical gap in the Python ecosystem for high-speed, multi-domain concentric body modeling, delivering results over 160 times faster than BEM alternatives for compatible geometries.

Software Design

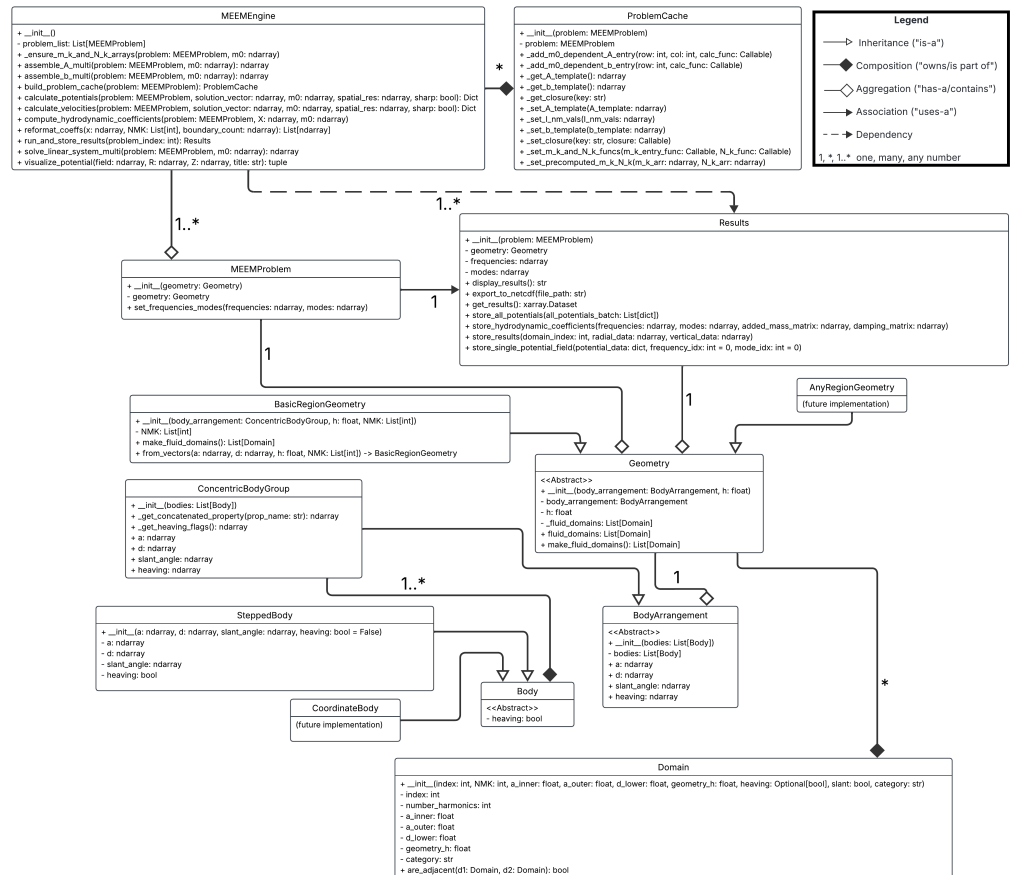


Figure 1: UML Diagram for OpenFLASH.

OpenFLASH balances computational efficiency with user accessibility through a tiered architecture. Figure 1 demonstrates the relationships between classes in the package. Users interact with high-level objects like SteppedBody and ConcentricBodyGroup, which partition the fluid volume into discrete Domain objects (see Figure 2). Figure 3 shows a typical problem geometry that is divided into multiple concentric fluid domains, including interior domains under the bodies and a final, semi-infinite exterior domain.

Table 1: Characteristics of Concentric Cylindrical Domains

Domains	Domain e (Exterior)	Domain i_1 (Interior 1)	Domains i_2 to i_J (Interior)
Top Boundary Condition	Wave surface	Body	Body
Bottom Boundary Condition	Sea floor	Sea floor	Sea floor
Radial Coordinate (r)	$r \rightarrow \infty$	$r = 0$	$0 < r < \infty$

Figure 2: A summary of the key attributes that define each type of fluid domain.

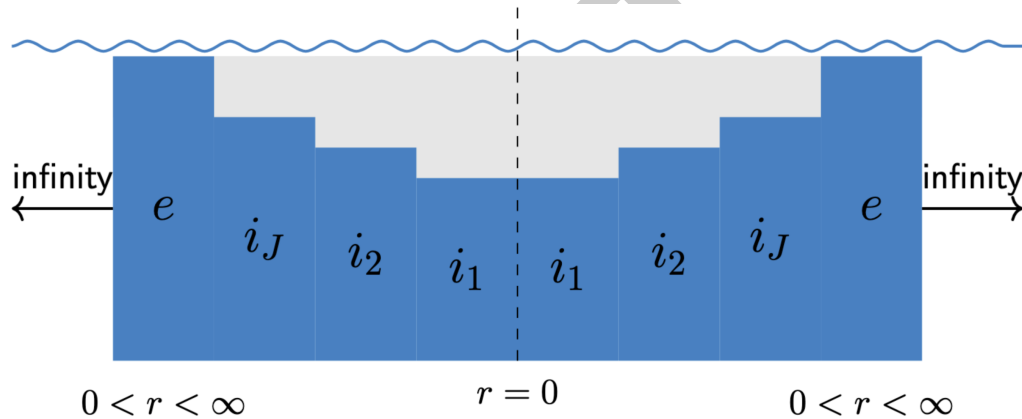


Figure 3: A typical problem geometry.

To minimize Python overhead during solving, the software employs an extraction engine that flattens these complex objects into raw NumPy arrays for high-performance linear algebra.

A key architectural trade-off was made in the ProblemCache class. To accelerate wide-frequency sweeps, we increased the memory footprint by precomputing frequency-independent matrix blocks and integration constants. This avoids rebuilding dense linear systems for every frequency step, a significant bottleneck in traditional solvers. Additionally, we opted to utilize xarray for data organization. While this adds a dependency, it ensures that OpenFLASH outputs are drop-in replacements for Capytaine users, facilitating immediate integration into existing research pipelines without requiring custom data parsers.

The package utilizes Sphinx to generate comprehensive documentation. The documentation includes a tutorial that guides users through the package's processes and explains its capabilities. The sphinx documentation is deployed in the browser through: <https://symbiotic-engineering.github.io/OpenFLASH/>.

A Streamlit application (docs/app.py) provides a graphical user interface for interacting with OpenFLASH. Users can define problem parameters through the GUI, run simulations, and visualize the resulting potential fields in real-time. This interactive tool enhances the usability and accessibility of the package. The streamlit app is deployed through: https://symbiotic-engineering.github.io/OpenFLASH/app_streamlit.html.

The package includes a comprehensive suite of unit tests (tests directory) using the pytest framework to ensure the code's reliability and correctness. These tests cover the core functionalities, ensuring the reliability and correctness of the code across different modules.

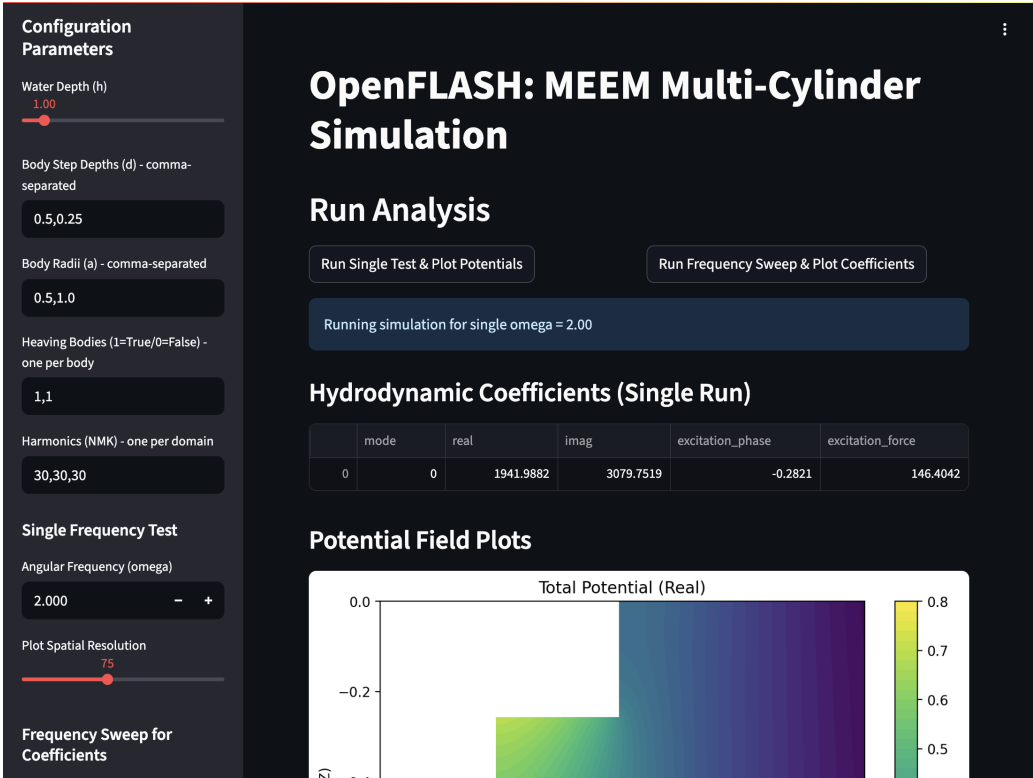


Figure 4: Streamlit Screenshot 1

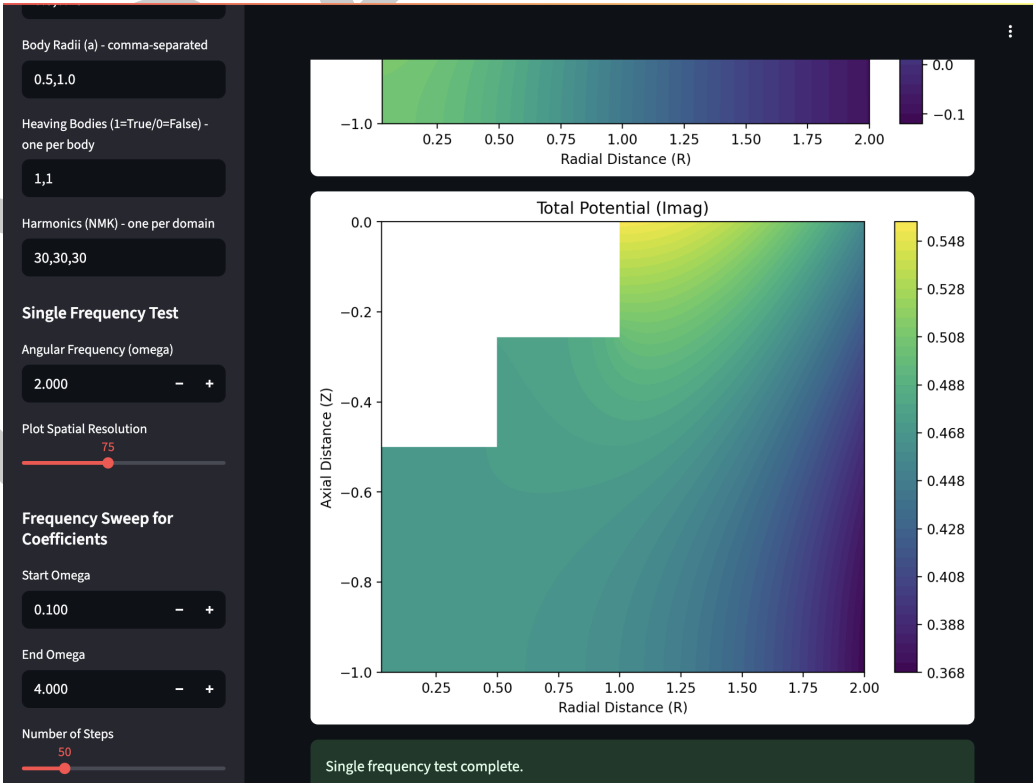
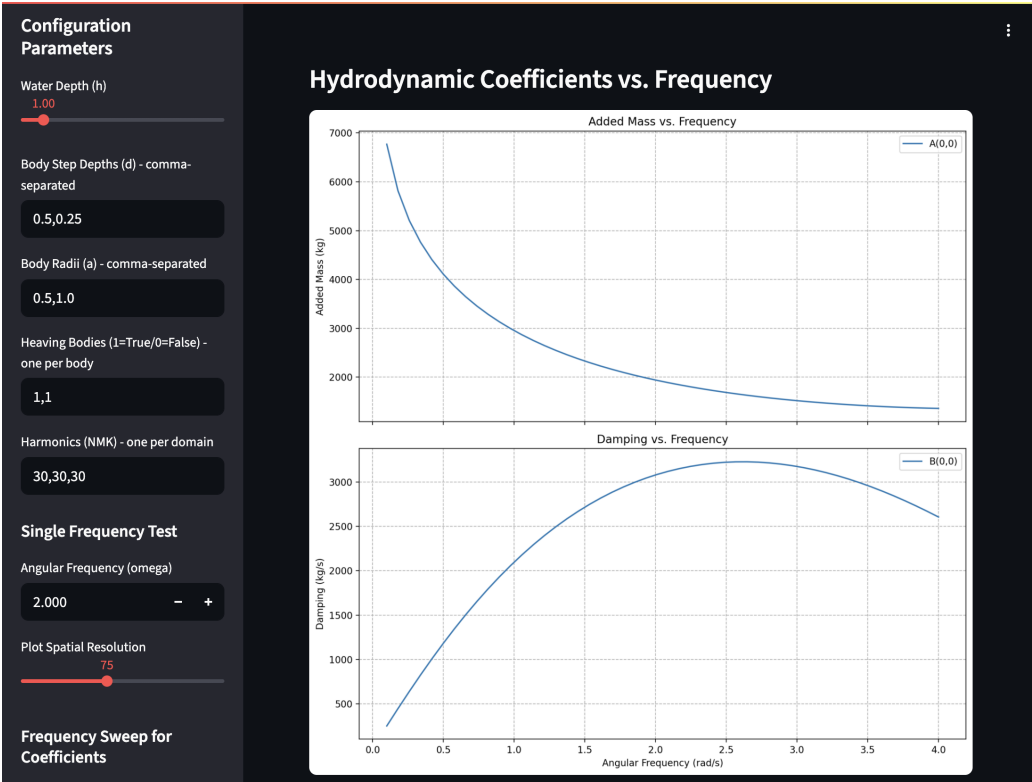


Figure 5: Streamlit Screenshot 2



Research Impact Statement

OpenFLASH provides a high-performance alternative to BEM solvers, optimized for iterative design. Its impact is already evidenced by its integration into MDOcean (McCabe et al., 2026), where its speed allows researchers to evaluate thousands of design iterations that were previously computationally prohibitive.

Comparative benchmarking against Capytaine (available in `package/test/benchmark_openflash_vs_capytaine`) demonstrates that OpenFLASH achieves 98.19% accuracy in hydrodynamic coefficient calculations while delivering a 162-fold reduction in runtime and requiring 1,000 times less memory. First presented at the UMERC+METS 2024 Conference, OpenFLASH represents the first open-source Python implementation of MEEM for multi-domain cylindrical structures. By lowering the barrier to entry for high-fidelity hydrodynamic modeling, it serves as a community-ready tool for advancing offshore renewable energy research.

AI Usage Disclosure

Generative AI tools, specifically Gemini 3 Pro and GitHub Copilot, were used to assist in drafting the initial structure of the documentation and to refactor parts of the ProblemCache logic. All AI-generated code was reviewed, integrated into the pytest suite, and verified against numerical baselines derived from previous analytical work. No AI tools were used in the writing of this manuscript.

Acknowledgements

We acknowledge the foundational work on MEEM as presented in the cited references. The development of OpenFLASH has been supported by the resources and expertise within the SEA Lab, led by Professor Maha N. Haji. We also acknowledge the contributions of the following students to the development of this software: Yinghui Bimali, En Lo, and John Fernandez. We also acknowledge the ongoing collaborations with Jessica Nguyen, Clint Chester Reyes, and Brittany Lydon for integrating the semi-analytical code they developed for other geometries in future OpenFLASH releases. We thank Prof. R. W. Yeung and Seung-Yoon Han for discussions on the theory and computation of this method.

We also gratefully acknowledge support for Kapil Khanal and Collin Treacy from a Sandia National Laboratories seedling grant and Rebecca McCabe from the NSF GRFP. The contributions of undergraduate researchers were supported by Cornell University's Engineering Learning Initiative (ELI).

This material is based upon work supported by the National Science Foundation Graduate Research Fellowship under Grant No.DGE-2139899. Any opinion, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation.

References

- Ancellin, M., & Dias, F. (2019). Capytaine: A python-based linear potential flow solver. *Journal of Open Source Software*, 4(36), 1341. <https://doi.org/10.21105/joss.01341>
- Bhattacharya, S., Pennock, S., Robertson, B., Hanif, S., Alam, M. J. E., Bhatnagar, D., Prezioso, D., & O'Neil, R. (2021). Timing value of marine renewable energy resources for potential grid applications. *Applied Energy*, 299, 117281. <https://doi.org/10.1016/j.apenergy.2021.117281>

- 113 Choi, Y.-M. (2019). *Two-way coupling between potential and viscous flows for a marine*
114 *application* (Theses No. 2019ECDN0046, École centrale de Nantes). [https://theses.hal.](https://theses.hal.science/tel-02493305)
115 [science/tel-02493305](https://theses.hal.science/tel-02493305)
- 116 Fusco, F., Nolan, G., & Ringwood, J. (2010). Variability reduction through optimal combination
117 of wind/wave resources – an irish case study. *Energy*, 35(1), 314–325. [https://doi.org/10.](https://doi.org/10.1016/j.energy.2009.10.018)
118 [1016/j.energy.2009.10.018](https://doi.org/10.1016/j.energy.2009.10.018)
- 119 McCabe, R., Dietrich, M., & Haji, M. (2026). Leveraging multidisciplinary design optimization
120 to advance wave energy converter viability. In *Renewable Energy*.
- 121 McCabe, R., Khanal, K., & Haji, M. (2024). Open-source toolbox for semi-analytical
122 hydrodynamic coefficients via the matched eigenfunction expansion method.
123 *UMERC+METs 2024 Conference*. <https://doi.org/10.5281/zenodo.14504016>
- 124 Olaya, S., Bourgeot, J.-M., & Benbouzid, M. E. H. (2015). Hydrodynamic coefficient
125 computation for a partially submerged wave energy converter. *IEEE Journal of Oceanic*
126 *Engineering*, 40(3), 522–535. <https://doi.org/10.1109/JOE.2014.2344951>

DRAFT